

EDGE: DBMS-Empowered Boolean Decomposition for GIG Synthesis

Ruofei Tang¹, Xuliang Zhu³, Xinyi Zhang¹, Lei Chen², Xing Li², Mingxuan Yuan², Jianliang Xu¹

¹ Department of Computer Science, Hong Kong Baptist University, Hong Kong, China

² Huawei Noah's Ark Lab, Hong Kong, China

³ Antai College of Economics and Management, Shanghai Jiao Tong University, Shanghai, China

{csrftang, xinhuang, xujl}@comp.hkbu.edu.hk, zhu.xl@sjtu.edu.cn, {lc.leichen, li.xing3, yuan.mingxuan}@huawei.com

Abstract—Boolean decomposition is a powerful technique in logic synthesis that breaks down Boolean functions into simpler components. Decomposition-based logic synthesis yields high-quality results and is particularly effective when combined with small-window optimization methods in Gate-Inverter Graphs (GIG). However, the efficiency limitations of current methods have constrained their applicability in handling large and complex logic. To address this challenge, we propose a novel framework, called *EDGE*, which leverages modern database techniques to accelerate Boolean decomposition, thereby achieving improved synthesis results while maintaining high efficiency. Experimental results demonstrate a runtime speedup of up to $21\times$ and an overall reduction in node count of at least 15% compared to state-of-the-art synthesis methods.

I. INTRODUCTION

The transformation from a two-level Sum-of-Product (SOP) to a multi-level logic is a key process that affects the area of synthesized circuits. The modern workflow of the transformation process, as shown in Figure 1①, simplifies the input SOP and optimizes it by extracting divisors [1]. From the optimized SOP, a Gate-Inverter Graph (GIG), a data structure used to represent multi-level logic implementations, is generated. A series of small-window optimization methods are subsequently applied to the GIG. This workflow is efficient and generally effective. However, it lacks control over the GIG structure, creating a gap between simpler SOPs and simpler GIGs. Despite some factoring methods intended to enhance logic sharing, these methods are coarse-grained and rely heavily on structural hashing for more localized sharing [2], which is insufficient. Moreover, internal Don't Cares (DCs) are ignored during the transformation and must be recomputed during small-window GIG optimization. To address these issues, Boolean decomposition has been proposed for the transformation [3].

Boolean decomposition is a powerful but time-consuming technique to break down a complex Boolean function into simpler sub-functions. For a given input subset, i.e. the bound set, it extracts a minimal sub-function from the original function, effectively utilizing Don't Cares (DCs) and discovering logic sharing. Figure 1② shows a typical decomposition-based transformation workflow, where the function is repeatedly decomposed until each sub-function is at the gate level, and is converted into a GIG. To improve efficiency, many approaches use single-output disjoint support decomposition (DSD) with binary decision diagrams (BDDs) [3], [4]. BDDs limit the solution space and ignore logic sharing among primary outputs which leads to larger GIGs. Additionally, gate-level decomposition on BDDs is unnecessary, as decomposing small functions is generally less effective compared to SOP-based transformation.

Considering the above limitations, we adopt multi-output Boolean decomposition and propose the transformation flow as shown in Figure 1③. We use multi-output decomposition to break down large functions into small-size sub-functions (≤ 6 inputs) and then synthesize them using the SOP-based workflow. This pipeline allows

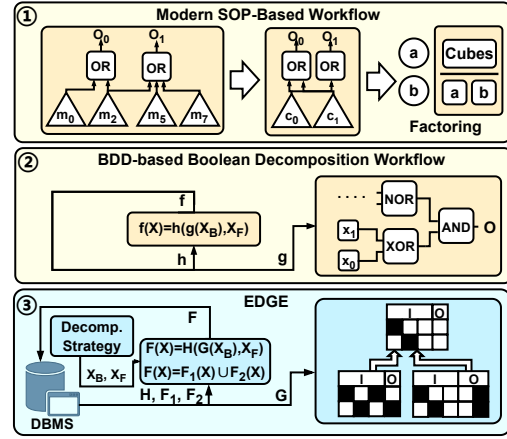


Fig. 1: Comparison of different transformation workflows.

EDGE is our proposed workflow.

us to control the overall multi-level structure while achieving effective GIG implementations for each sub-function. To realize this workflow, we face two main challenges:

1. Multi-output Boolean decomposition faces efficiency challenges due to its NP-hard nature, which limits the size of functions to be processed. Existing solutions use set algebra to form the decomposition process [5], [6], but this approach is difficult to implement and fails to address the runtime problem.
2. The solution space for decomposition grows exponentially with the size of the input set. To the best of our knowledge, there is no framework that can effectively make decisions for a long decomposition chain.

To tackle these challenges, we present the following contributions and results:

- We improve the efficiency of Boolean decomposition using a Database Management System (DBMS). By formulating the decomposition as relational query operations, we optimize its time complexity and leverage various DBMS techniques to accelerate the process.
- We propose a novel Boolean decomposition-based DBMS-accelerated GIG synthesis framework, *EDGE*, to manage the continuous decision process of decompositions and schedule the synthesis of each sub-function.
- We propose multiple decomposition strategies for different types of functions. To implement these strategies, we design two heuristic output clustering algorithms for parallel decomposition and introduce an efficient bound set search method for filtering high-quality decompositions.
- We conduct extensive evaluations on *EDGE*. The results demon-

strate that our framework achieves up to 21× speedup in multi-output decomposition compared to existing methods, while reducing the total GIG size by at least 15% compared to state-of-the-art (SOTA) SOP-based transformation methods.

II. PRELIMINARIES

A. Definitions

Boolean Functions. A Boolean function maps a set of Boolean variables to a single Boolean output, denoted as $y = f(X)$, where y is the *output variable*, f is the *Boolean function*, and $X = \{x_0, x_1, \dots\}$ is the set of *input variables*. When multiple Boolean functions share the same input set X , they can be represented as $Y = F(X)$, where $Y = \{y_0, y_1, \dots\}$ is the set of output variables and $F = \{f_0, f_1, \dots\}$ is the corresponding set of Boolean functions. In this case, F is called a *multi-output Boolean function*. For the remainder of this paper, we use “Boolean functions” to refer to both single-output and multi-output Boolean functions.

Truth Tables. A truth table records the input-output relationships of Boolean functions in a tabular form. A specific combination of input or output values is called a *pattern*. In logic synthesis, due to the presence of *don’t cares* (DCs) in designs, a truth table may omit some input patterns, resulting in an *incompletely specified* truth table. **Sum of Products (SOP)** is a compact representation of a truth table with DC values in input and output combinations. Each row in the SOP is called a cube.

Boolean Network. A Boolean network is a directed acyclic graph (DAG) representing Boolean functions [7]. A **Gate-Inverter Graph (GIG)** is a type of Boolean network. A GIG consists of a single type of logic component, and each node has a fixed in-degree (typically 2). Logic NOT gates can be placed on the edges. Common types of GIGs include AND-Inverter Graphs (AIGs), XOR-Inverter Graphs (XIGs), and Majority-Inverter Graphs (MIGs) [8].

Boolean Decomposition. Boolean decomposition is a technique used to break down a Boolean function into simpler sub-functions. Given a Boolean function $Y = F(X)$, there are two types of Boolean decomposition operations [5]:

- *Serial Decomposition.* A serial decomposition extracts a sub-function from a subset of input variables. The decomposition $Y = H(G(X_b), X_f) = F(X)$ is a serial decomposition of F , where $X = X_b \cup X_f$, X_b is the *bound set*, X_f is the *free set*. H and G are sub-functions of Y , and the set $X_s = X_b \cap X_f$ is called the *shared set*. If $X_s = \emptyset$, the decomposition is a *disjoint support decomposition* (DSD). G can be a multi-output Boolean function, and its number of output variables is denoted as $|G|$. For a bound set X_b and a free set X_f in the same serial decomposition, we refer to (X_b, X_f) as a *bound-free pair*.
- *Parallel Decomposition.* A parallel decomposition splits the output set into new sub-functions. If $Y = Y_0 \cup Y_1 \cup \dots$ is a disjoint split of Y , where $Y_i \cap Y_j = \emptyset$ for any $i \neq j$, then $Y_0 = F_0(X), Y_1 = F_1(X), \dots$ is a parallel decomposition of Y , where $F_0, F_1, \dots$ are sub-functions of F .

Relational Algebra Operators. The relational algebra operators used in this paper are listed below [9]:

- σ_θ (select): filtering rows in a relation based on the specified condition θ .
- $\times$ (cartesian-product): producing a relation that is the combination of every tuple in two relations.
- Π_A (project): extracting the specified column set A from a relation, with duplicates removed.

- $\bowtie_\theta$ (join): combining tuples from two relations based on a matching condition θ .

B. Related Works

The inspiration for this work primarily comes from multi-output Boolean decomposition on SOPs [5], [6] and support-reducing decomposition synthesis [3], [4].

In the studies on multi-output decomposition, two basic operations were defined for SOPs: serial decomposition and parallel decomposition [5], [6]. To unify operations on DCs and 1s and 0s, complex set systems were used to build the serial decomposition process, making it difficult to implement and resulting in inefficiency. However, in practice, simplified SOPs do not always have significantly fewer rows than truth tables. Thus, for ease of implementation and optimization of efficiency, we adopt truth tables instead of SOPs. Additionally, no prior work provided a decomposition strategy suitable for modern-scale functions, which motivated us to design **EDGE**.

In the study on BDD-based DSD for GIG small-window optimization [3], the potential of Boolean decomposition to discover good GIG structures was demonstrated. However, limited by BDD, an inefficient and complex process for logic sharing was required, restricting the window size to 30 nodes and 8 inputs, which reduced the overall effectiveness. In another study [4], the same technique was applied on a larger scale, focusing on reducing the LUT count after mapping rather than optimizing the GIG area. In the process, small-window optimizations were combined to improve the quality of results, but single-output decompositions ignore logic sharing among primary outputs, limiting the effectiveness.

Based on the strengths and addressing the weaknesses of these existing works, we aim to accelerate multi-output Boolean decomposition and extend its applicability to 16 inputs, thereby achieving a better structure on a larger scale and reducing overlap with small-window optimization for improved results.

This work is also influenced by other decomposition-related algorithms, including LUT network optimization [10]–[12] and decomposition method acceleration [13]–[15].

III. EDGE

EDGE is a novel framework that utilizes Boolean decomposition for logic synthesis, integrating a DBMS [9] to facilitate large-scale multi-output Boolean decompositions. However, as the scale of processed Boolean functions increases, the solution space grows exponentially, posing substantial challenges to scalability improvements. Therefore, a framework is necessary to evaluate solutions, reduce the search space, and manage the synthesis process. The architecture of **EDGE**, as depicted in Figure 2, is specifically designed to meet these requirements. Below, we outline the workflow of **EDGE** and provide detailed explanations of each component involved.

Workflow. The input truth table is initialized as a Truth Table Unit, which then becomes the root unit node of the Decision Tree, denoted as U_0 , and is recorded in the Edge Node Pool. The Decomposition Manager starts working by extracting U_0 from the pool and finding potentially valuable decompositions for the truth table. The selected operations are applied to the unit to generate sub-functions and update the Decision Tree.

The sub-functions are then initialized as new Truth Table Units. If a sub-function’s input set size is below a predefined threshold, the unit is marked as terminal and passed to an external GIG synthesis tool [16] for implementation. Non-terminal nodes are added to the Edge Node Pool, and the process repeats until the pool is empty and all edge nodes in the Decision Tree are terminal.

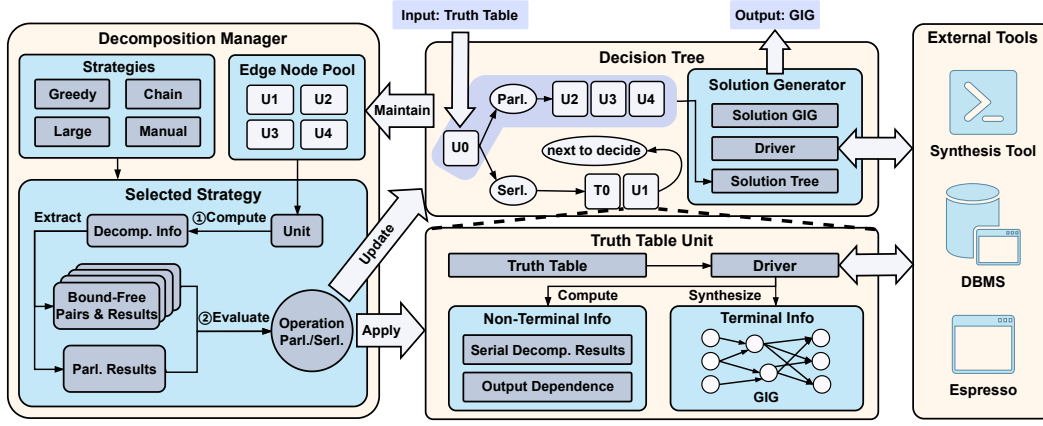


Fig. 2: Architecture of *EDGE*. Parl. and Serl. are short for parallel and serial decomposition operation nodes. T for terminal nodes, U for truth table unit nodes.

The Decision Tree can record multiple decomposition-based synthesis solutions, each corresponding to a subtree known as a Solution Tree. By using Solution Trees, the GIGs of sub-functions can be composed, with small-window optimizations performed via the external synthesis tool, resulting in the final GIG output.

Truth Table Unit. The Truth Table Unit serves as the foundational structure in *EDGE*. It stores the truth table and provides interfaces for decomposition operations and GIG implementation. The truth table is managed within the DBMS, which greatly reduces memory usage and accelerates decompositions. Espresso [17] is used to compute each output's support set for clustering during parallel decomposition.

A unit is marked as terminal when its truth table size is below a specified threshold. This approach is based on the observation that combinations of GIG small-window optimization methods can achieve near-optimal implementation for truth tables with up to 6 inputs in a very short time.

Decision Tree. The Decision Tree serves two main purposes: tracking the synthesis process and generating the final GIG output. Each truth table unit is considered a unit node in the tree.

For a function, there are many possible decomposition paths; the Decomposition Manager strategically selects a small subset of these paths to minimize the solution space. After each decomposition, multiple sub-functions are produced, and the Decision Tree records this entire process.

A sub-tree of the Decision Tree, called the Solution Tree, tracks a complete decomposition-based synthesis process. By traversing the operation nodes of a solution tree in the reverse topological order, the solution GIG is composed. Finally, the GIG implementation of the root node is obtained. Each solution tree yields a different root GIG, and the optimal one is selected as the final output.

Decomposition Manager. The Decomposition Manager determines the decomposition type and details (bound set, free set, or output split) for the nodes from the Edge Node Pool.

Different types of Boolean functions require different strategies to ensure high-quality decomposition. The strategy begins by testing a set of potential decomposition results. These results are then evaluated to determine the final operations. Multiple algorithms are proposed to support the strategies. Details are provided in Section IV.

IV. TECHNIQUES

A. Database-Accelerated Serial Decomposition

Serial decomposition is the core algorithm in *EDGE* and is executed many times within a transformation process, thus requiring

high efficiency. However, it involves solving an NP-hard problem, and the demand for accuracy makes it hard to approximate the process. Existing works rely on complex set systems, which are challenging to implement and introduce unnecessary computation. To tackle these problems, we use relational algebra query operators to build the algorithm. The algorithm is divided into two stages: Prepare and Commit. For a serial decomposition $Y = F(X) = H(G(X_b), X_f)$, the Prepare phase computes the G sub-function, and the Commit phase computes the H sub-function.

Prepare Phase. The Prepare phase is illustrated in Algorithm 1. Before explaining the process, we first introduce several key concepts and a lemma:

- **Bound / Free / Output Pattern.** The pattern represents a specific set of Boolean variable assignments for the bound set, free set, and output set, respectively.
- **Conflict Pair.** A conflict pair is defined as a tuple of two bound patterns, $\langle X_{bi}, X_{bj} \rangle$, derived from two rows in a truth table, denoted as r_0 and r_1 . If r_0 and r_1 have the same free pattern but different output patterns, then the bound patterns of r_0 and r_1 are said to be in *conflict*, forming a conflict pair.

Lemma 1 Given a serial decomposition $F(X) = H(G(X_b), X_f)$, the decomposition is valid if and only if, for every conflict pair $\langle X_{bi}, X_{bj} \rangle$, it holds that $G(X_{bi}) \neq G(X_{bj})$.

Proof Assume $F(X) = H(G(X_b), X_f)$ is a valid decomposition, where there exists at least one conflict pair $\langle X_{bi}, X_{bj} \rangle$ such that $G(X_{bi}) = G(X_{bj})$. By the definition of a conflict pair, X_{bi} and X_{bj} have the same free pattern, i.e., $X_{fi} = X_{fj}$. Since $G(X_{bi}) = G(X_{bj})$ and $X_{fi} = X_{fj}$, it follows that: $H(G(X_{bi}), X_{fi}) = H(G(X_{bj}), X_{fj})$. This contradicts our definition of the conflict pair. Thus, our initial assumption must be false, proving Lemma 1.

Lemma 1 provides the condition for a valid serial decomposition. Thus, the first task in Algorithm 1 is to identify conflict pairs (line 1). According to the definition of conflict pairs, we can write the following relational algebraic expression to find them:

$$C \leftarrow \Pi_{F_1.Bound, F_2.Bound} (\sigma_{F_1.Free = F_2.Free} (\sigma_{F_1.Output \neq F_2.Output} (F_1 \times F_2))) \quad (1)$$

where F_1, F_2 are aliases of F , and *Bound*, *Free*, *Output* are short for bound, free and output patterns. This is an equivalent method to the SOP-based one in the previous work [6]. Given X as the input set, this query has a time complexity of $O((2^{|X|})^2)$. Although existing

Algorithm 1 Serial Decomposition Phase Prepare

Input: Truth Table F , Bound Set X_b , Free Set X_f **Output:** Truth Table G

- 1: Conflict Pairs $C \leftarrow \text{LookupConflictPairs}(F, X_b, X_f)$;
 - 2: Graph $\mathcal{G}_C \leftarrow (\Pi_{X_b}(F), C)$; $\triangleright \mathcal{G}_C : (\text{Nodes}, \text{Edges})$
 - 3: Map $M \leftarrow \mathcal{G}_C.\text{Coloring}()$; $\triangleright M : \text{Bound Pattern} \rightarrow \text{Color ID}$
 - 4: Map $P \leftarrow \text{AssignPattern}(M)$; $\triangleright P : \text{Color ID} \rightarrow \text{New Pattern}$
 - 5: $G \leftarrow M \bowtie P$; $\triangleright M$ joins P on Color ID
 - 6: **return** G ;
-

Algorithm 2 Bound-Free Set Pair Search

Input: Truth Table F , Flag *allow_share*, Search Range $[l, u]$ **Output:** Map M $\triangleright M$: bound set size $\rightarrow$ truth table array

- 1: Let X, Y be the input and output variable set of F ;
 - 2: Set $C \leftarrow \bigcap_{y \in Y} \text{Supports}(y)$;
 - 3: Integer $\langle \text{lower}, \text{upper} \rangle \leftarrow \langle l, \text{Min}(u, |C| - 1) \rangle$;
 - 4: **for** integer $k \in [\text{lower}, \text{upper}]$ **do**
 - 5: **for** set $X_b \in \text{Combinations}(C, k)$ **do**
 - 6: Truth Table $G \leftarrow \text{SerialPrepare}(F, X_b, X \setminus X_b)$;
 - 7: $M[k] \leftarrow M[k] \cup \{ \langle G, X_b, X \setminus X_b \rangle \}$;
 - 8: $M[k].\text{sort}()$;
 - 9: **if** *allow_share* **then**
 - 10: $\text{ExtendSharedSet}(M)$;
 - 11: **return** M ;
-

methods are applied to cubes, they still rely on expanding DCs to identify conflict pairs, resulting in the same time complexity.

To improve efficiency, we propose the following optimizations:

1. **Join Rewriting.** By adjusting the selection order, we can rewrite the cartesian-product into a conditional join, which greatly reduces the operation cost.
2. **Symmetry Elimination.** Due to the symmetry in conflict relations and output pattern comparison, we can add constraints to the inner join and replace $\neq$ with $<$ in the selection, which halves the rows to scan.

After these revisions, we refine Equation (1) as follows:

$$C \leftarrow \Pi_{F_1.\text{Bound}, F_2.\text{Bound}} \left(\left(\sigma_{F_1.\text{Output} < F_2.\text{Output}} \left(F_1 \bowtie_{F_1.\text{Free} = F_2.\text{Free} \wedge F_1.\text{Bound} < F_2.\text{Bound}} F_2 \right) \right) \right) \quad (2)$$

With these optimizations, the time complexity is reduced to $O(2^{\frac{2|X_b|+|X_f|}{2}})$. Additionally, executing the query in a DBMS provides several benefits [9]:

- **Sort-merge join.** For $F_1 \bowtie F_2$, the DBMS would use a sort-merge join rather than a double full table scan.
- **Multi-thread acceleration.** The DBMS splits the query operation into multiple independent subtasks and accelerates them through multithreading.
- **Storage optimizations.** These techniques are optimized for handling table data, reducing memory usage, improving storage efficiency, and speeding up queries.

After finding conflict pairs in Algorithm 1, we construct a valid minimal sub-function using graph coloring (lines 2-3). Each node represents a unique bound pattern, and each edge represents a conflict. After coloring, each color corresponds to a unique input pattern in the new function G . We assign each color a distinct pattern and construct G based on the coloring results (lines 4-5). Since graph coloring has near-linear time solutions under edge constraints [18], it is not a significant factor in efficiency.

Commit Phase. In the Commit Phase, we compute the sub-function H according to the sub-function G . Since G is a mapping from the X_b as a subset from X , then to obtain H , all we need to do is a

natural join of G and F on X_b , denoted as $H = G \bowtie F$, which is a very fast process.

B. Search of Bound-Free Pair

In Section IV-A, we improve the efficiency of serial decomposition. However, for a truth table with $|X|$ inputs, there can be $O(2^{|X|})$ possible bound-free pairs, even for DSDs. To avoid the overhead of exhaustive search, we present Algorithm 2.

The search is conducted using the bound set length as the unit. In line 1, we find the intersection of output supports, as including output-irrelevant inputs is meaningless. In lines 4-7, we iteratively test the results of serial decomposition on bound-free pairs within the search range and store the result tuples in M . In line 8, we sort the results based on two metrics of G within the same bound set size: (1) the number of outputs, and (2) the number of unique output patterns. We retain only the top- K tuples in M , where K is a hyperparameter. In lines 9-10, an optional step allows for extending the free set to form shared sets.

Shared sets can enhance connections between sub-functions, benefiting GIG optimization. However, searching for suitable shared sets is costly, so it is provided as an optional action, executed only when time permits. The heuristic algorithm for extending the free set of a DSD is as follows:

- S1.** Denote the result tuple to extend as $\langle G, X_b, X_f \rangle$. Initialize a set $S = \{ \{x\} \mid x \in X_b \}$.
- S2.** Pop X_S from S , and get $G_S \leftarrow \text{SerialPrepare}(F, X_b, X_f \cup X_S)$. If $|G_S| = 1$ or $|G_S| - |X_S| \leq |G|$, update $T \leftarrow T \cup \{ X_S \cup \{x\} \mid x \in X_b \setminus X_S \}$. Keep a record of the largest $X_S \in S$ as X_L .
- S3.** Repeat Step 2 until $S = \emptyset$. Return $X_L \cup X_f$.

Note that when the bound set size approaches the input size, the extension of free sets is less effective.

C. Heuristic Clustering of Parallel Decomposition

Parallel decomposition separates outputs with weak logical connections when serial decomposition struggles to find meaningful bound-free pairs, enabling the synthesis process to continue. We design two methods for parallel decomposition. The first method clusters output variables based on their support sets, and if this fails, we apply a second method that clusters outputs by unique pattern numbers.

By Support Set. When the support sets of a group of output variables differ significantly, serial decomposition will not yield meaningful results. To address this, we cluster the outputs based on their support sets. The Jaccard distance is used to measure the distance between two outputs:

$$D(y_i, y_j) = 1 - \frac{|\text{Supports}(y_i) \cap \text{Supports}(y_j)|}{|\text{Supports}(y_i) \cup \text{Supports}(y_j)|} \quad (3)$$

where $y_i, y_j \in Y$, $D(y_i, y_j)$ is the distance between y_i and y_j . Hierarchical clustering with average linkage is then used to adaptively determine the number of clusters.

By Unique Pattern. The process for this method is shown in Algorithm 3. The function $\text{COUNT}(query)$ returns the number of rows in the query result. The goal is to minimize unique output patterns in the two sub-functions.

D. Decomposition Strategies

Decomposition strategies determine the appropriate decomposition for each input truth table after search and evaluation. To handle functions with various characteristics, we propose three automatic

Algorithm 3 Unique Pattern Based Parallel Decomposition

Input: Truth Table F **Output:** Cluster C_0, C_1

```

1: Let  $Y$  be the output variable set of  $F$ ;
2: for  $y \in Y$  do
3:   if  $\frac{\text{COUNT}(\Pi_{C_0 \cup \{y\}}(F))}{\text{COUNT}(\Pi_{C_0}(F))+1} \leq \frac{\text{COUNT}(\Pi_{C_1 \cup \{y\}}(F))}{\text{COUNT}(\Pi_{C_1}(F))+1}$  then
4:      $C_0 \leftarrow C_0 \cup \{y\}$ ;
5:   else
6:      $C_1 \leftarrow C_1 \cup \{y\}$ ;
7: return  $C_0, C_1$ ;

```

Algorithm 4 Greedy Strategy

Input: Truth Table F , Termination Size t , Search Range r **Output:** Operations Op

```

1:  $F_R \leftarrow \text{RemoveTrivialOutputs}(F)$ ;
2: Let  $X, Y$  be the input and output variable set of  $F_R$ ;
3: if  $|X| \leq t$  then ▷ Termination condition
4:    $Op \leftarrow \text{Terminate}$ ;
5:   return  $Op$ ;
6:  $Results \leftarrow \text{SerialPairSearch}(F, \text{True}, [|X| - r, |X| - 1], k)$ ;
7:  $Results \leftarrow Results \cup \text{SerialPairSearch}(F, \text{True}, [2, 2 + r], k)$ ;
8: if  $|Y| = 1$  then ▷ Single-output situation
9:    $Op \leftarrow \text{EvaluateSingle}(Results)$ ;
10:  if  $Op = \emptyset$  then
11:     $Op \leftarrow \text{EarlyTerminate}$ ;
12: else if  $|Y| > 1$  then ▷ Multi-output situation
13:    $Op \leftarrow \text{EvaluateMultiple}(Results)$ ;
14:   if  $Op = \emptyset$  then
15:      $Op \leftarrow \text{HeuristicParallel}(F)$ ;
16: return  $Op$ ;

```

strategies: Greedy, Large, and Chain. Additionally, we provide a **Manual** mode for debugging and developing new strategies.

Greedy. The greedy strategy is designed to handle general cases. It prioritizes support-reducing serial decomposition, with parallel decomposition applied only if no good bound-free pair is found. The process is shown in Algorithm 4.

In line 1, `RemoveTrivialOutputs` preprocesses F by removing output variables whose support set contains 2 or fewer inputs, as these are easy to implement and could interfere with finding meaningful bound-free pairs. In lines 2-4, we check if the termination condition is met. If so, the `Terminate` operation is returned, and the truth table is passed to the GIG synthesis tool. In lines 6-7, bound-free pairs are searched within the specified range from both sides. Lines 8-15 evaluate single-output and multi-output Boolean functions using different criteria to decide the final operation. Elements of $Results$ are denoted as $\langle G = Y_G(X_b), X_b, X_f \rangle$ for subsequent use.

For single-output Boolean functions, the results of serial decompositions are first filtered based on the following conditions:

- **Not Support-reducing:** $|G| = |X_b|$.
- **No Logic Sharing:** $\text{COUNT}(\Pi_{Y_G}(G)) = 2^{|X_b|}$.

The second condition comes from Shannon Decomposition:

$$f(X, x_i) = x_i \cdot f(X, 1) + \bar{x}_i \cdot f(X, 0) \quad (4)$$

which represents the worst-case result of a serial decomposition without any logic sharing, refined here to the pattern level.

If no valid tuple remains after filtering, `EarlyTerminate` is returned for F even if $|X| > t$. For the remaining decompositions, the following criteria are used to determine the best option, in order of priority:

1. **Is Single-Output:** $|G| = 1$.
2. **Estimated Area Sum:** $|G| \cdot 2^{|X_b|} + |F_R| \cdot 2^{|G|+|X_f|}$.

Algorithm 5 Chain Strategy

Input: Chained Truth Table F_C **Output:** Operation Op

```

1: Let  $X, Y$  be the input and output variable set of  $F_C$ ;
2:  $y_0 \leftarrow \arg \min_{y \in Y} |\text{Supports}(y)|$ ;
3: if  $|\text{Supports}(y_0)| \leq 2$  then
4:    $Op \leftarrow \text{ParallelByGroup}(Y \setminus \{y_0\}, \{y_0\})$ ;
5: else
6:    $Op \leftarrow \text{SerialPrepare}(F, \text{Supports}(y_0), X \setminus \text{Supports}(y_0))$ ;
7: return  $Op$ ;

```

3. **Bound Set Scale:** $\text{COUNT}(\Pi_{Y_G}(G))$.

For multi-output decompositions, similar steps are followed, but with differences in filtering conditions and evaluation criteria:

- **No Logic Sharing:** $|G| = \sum_i |G_i| \wedge \text{COUNT}(\Pi_{Y_G}(G)) = 2^{|G|}$, where $G_i = \text{SerialPrepare}(\Pi_{X, \{y_i\}}(F), X_b, X_f)$, $y_i \in Y$.

If no valid tuple remains after filtering, a parallel decomposition is selected as the operation on F . The evaluation criteria are similar to the single-output case, but **Is Single-Output** and **Estimated Area Sum** are modified as follows:

- **Is Single-Output:** $|G| \leq |F|$.
- **Estimated Area Sum:** $\frac{|G|}{\sum_i |G_i| - |G|} \cdot 2^{|X_b|} + |F_R| \cdot 2^{|G|+|X_f|}$.

Here, an extra bias is introduced to prioritize logic sharing.

Large. The Large strategy is a variant of the greedy strategy, designed to reduce the runtime for large truth tables by avoiding searches of large bound sets. Adjustments include:

1. Bound-free pairs are searched only from the lower side instead of both sides (line 7 in Algorithm 4).
2. Pairs are searched step by step rather than over a range. Once a feasible serial decomposition is found, it is returned.
3. If the bound set size becomes large (≥ 12), `EarlyTerminate` or `HeuristicParallel` is selected as the operation.

These modifications may impact result quality, and users can choose the strategy according to the specific scenario.

Chain. The Chain strategy is designed for a special type of functions where $\text{Supports}(y_0) \subseteq \text{Supports}(y_1) \subseteq \text{Supports}(y_2) \subseteq \dots$ with all outputs logically related. The optimal structure of such functions forms a chain. In practice, these functions are identified based on their output support sets. The process of the chain strategy is shown in Algorithm 5. There is no need to search for pairs, as the optimal solution is obvious. Once the bottom-level output becomes trivial, it is separated from the function.

V. EXPERIMENTS

EDGE is implemented in Python, using *DuckDB* [19] as the DBMS and *NetworkX* [20] for graph algorithms support. All experiments are conducted on a server with an AMD EPYC 7742 CPU and on IWLS 2023 contest benchmarks.¹

A. Efficiency of Serial Decompositions

This subsection evaluates the efficiency of our serial decomposition implementation under various settings. The original method based on Equation (1) could not complete within 3,600 seconds in our experiments. Consequently, we compare our approach exclusively with the complexity-reduced method based on Equation (2) without DBMS acceleration, which we will refer to as the baseline.

EXP-1: Comparison on 16-Input Truth Tables. In this experiment, we compare the total runtime of serial decomposition for 16-input

¹https://www.iwls.org/iwls2023/2023_IWLS_Contest_Benchmarks.zip

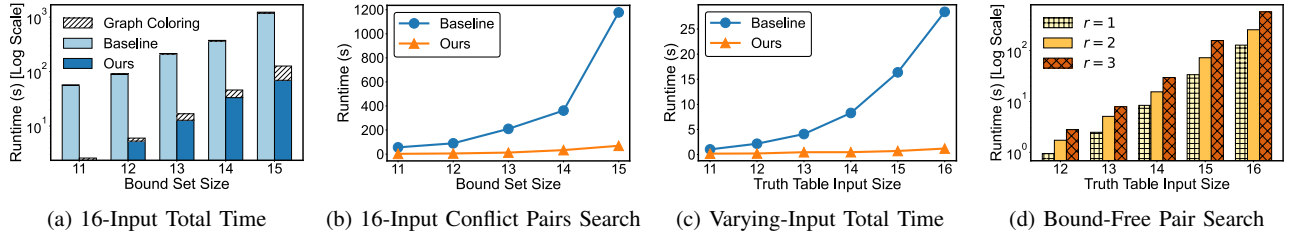


Fig. 3: Efficiency of Serial Decomposition

truth tables with different bound set sizes. In Figure 3(a), “Baseline” and “Ours” represent the runtime of the conflict pairs search component in the baseline method and our approach, respectively, while “Graph Coloring” denotes the runtime associated with graph coloring. As can be seen, the pairs search component dominates the overall runtime. Our proposed approach achieves a reduction in the runtime of this component by 94-95% compared to the baseline method across different bound set sizes, as shown in Figure 3(b). Consequently, the total runtime using our approach is 10-21 $\times$ better than that of the baseline method.

EXP-2: Comparison for Fixed Bound Set Size. In this experiment, we compare the runtime for different input sizes with a fixed bound set size of 10. Figure 3(c) shows the runtime trend as the input size increases. While the initial improvement in runtime is 10 $\times$ at an input size of 11, this enhancement grows exponentially as the input size increases, proving the scalability of our optimizations.

EXP-3: Evaluation of Bound-Free Pair Search. This experiment evaluates the efficiency of bound-free pair search with varying truth table sizes and different bound-free pair search ranges. We use 32 parallel threads to accelerate the search, without applying the Large strategy. Figure 3(d) shows that for a truth table with 16 inputs, the total runtime with $r = 3$ is the longest, reaching 576 seconds, while with $r = 1$, it takes only 127 seconds. This difference arises from two factors: (1) the number of bound-free pairs within the range, and (2) the runtime of decompositions for different bound set sizes. For a 12-input truth table, reducing 4 inputs results in a runtime of only 2.7 seconds for $r = 3$. This highlights the rationale behind our Large strategy: to quickly reduce truth table size while focusing on decomposition quality for smaller functions.

B. Effectiveness of *EDGE*

EXP-4: Comparison of Synthesis Results. In this experiment, we evaluate and compare the GIG synthesis results of *EDGE* against SOTA SOP-based transformation processes. All input truth tables are considered random, with no specific optimizations for any instance. The termination size is set to 6 for truth tables with more than 8 inputs and 4 otherwise. The benchmarks are grouped based on the official information from the IWLS 2023 contest, and all synthesis results are verified by comparing the simulated truth tables. The quality of the synthesis is measured by the number of AIG nodes. We compare three methods implemented in ABC [16]. The sequence used for all methods includes reading truth tables, simplifying SOPs, and converting to AIGs: *read_truth -xf; collapse; sop; [factorization command]; strash;*. Original does not use a factorization command, while *fx* and *fxch* use *fx* and *fxch* commands, respectively.

Table I shows the node counts of AIGs after transformation. For each dataset, *EDGE* produces fewer initial AIG nodes than other methods, achieving an 18% reduction in the total node count compared to the best SOTA (*fx*) results. Although having fewer initial AIG nodes does not necessarily mean fewer nodes after

TABLE I: Number of Nodes After Transformation

Benchmarks	Original	<i>fx</i>	<i>fxch</i>	<i>EDGE</i>
ex00-ex01	1,018	736	795	654
ex02-ex07	4,626	3,229	3,494	3,074
ex08-ex09	4,384	2,660	2,817	2,452
ex10-ex15	5,978	3,833	3,902	3,523
ex16-ex27	20,171	13,722	14,266	11,469
ex28-ex49	19,569	13,199	14,424	12,351
ex50-ex67	37,561	23,322	25,211	16,526
ex68-ex99	18,310	12,521	13,510	9,986
Total	111,617	73,222	78,419	60,035

TABLE II: Number of Nodes After GIG Optimization

Benchmarks	Original	<i>fx</i>	<i>fxch</i>	<i>EDGE</i>
ex00-ex01	704	669	704	602
ex02-ex07	3,408	2,959	3,147	2,878
ex08-ex09	2,958	2,474	2,574	2,364
ex10-ex15	4,122	3,443	3,485	3,162
ex16-ex27	13,984	12,571	13,007	10,787
ex28-ex49	13,341	11,893	12,658	11,662
ex50-ex67	25,704	21,313	22,791	15,739
ex68-ex99	12,489	10,936	11,649	9,040
Total	76,710	66,258	70,015	56,234

optimization, a smaller initial size reduces the runtime of powerful but time-consuming optimization methods [3], [21]. The average runtime for *EDGE* per truth table is **230** seconds. Notably, for truth tables with 14 or more inputs and multiple outputs, the primary source of runtime is the Espresso computation for output supports.

Table II shows the node counts of AIGs after transformation and optimization using a sequence of instructions: *fraig; resyn; resyn2; resyn2a; resyn3; drw; dc2; refactor;*, repeated 50 times to approach the optimization limit. Our method outperforms all SOTA methods across all datasets, achieving a 15% reduction in total node count compared to the best SOTA (*fx*) results. In 36 specific test cases, our method achieves more than a 10% reduction compared to the best SOTA results.

VI. CONCLUSION

In this work, we propose a two-level logic transformation process based on multi-output Boolean decomposition to enhance the quality of GIG synthesis. To improve efficiency, we have formulated the process as relational query operations and leverage DBMS techniques to accelerate computations and reduce redundancy. For better synthesis quality, we have proposed a decomposition framework, *EDGE*, with tailored strategies to handle truth tables of varying characteristics. Experimental results demonstrate that *EDGE* achieves significant improvements in decomposition efficiency and higher-quality synthesis results compared to SOTA methods.

Acknowledgment. This work is partially supported by the Hong Kong Research Grants Council and Guangdong Basic and Applied Basic Research Foundation (Project No. 2023B1515130002). Lei Chen is the corresponding author.

REFERENCES

- [1] B. d. O. Schmitt, A. Mishchenko, V. N. Kravets, R. K. Brayton, and A. I. Reis, "Fast-extract with cube hashing," in *2017 22nd Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2017, pp. 145–150.
- [2] A. Mishchenko, S. Chatterjee, R. Jiang, and R. K. Brayton, "Fraigs: A unifying representation for logic synthesis and verification," ERL Technical Report, Tech. Rep., 2005.
- [3] L. Machado and J. Cortadella, "Boolean decomposition for aig optimization," in *Proceedings of the on Great Lakes Symposium on VLSI 2017*, 2017, pp. 143–148.
- [4] —, "Support-reducing decomposition for fpga mapping," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 1, pp. 213–224, 2018.
- [5] T. Łuba and H. Selvaraj, "A general approach to boolean function decomposition and its application in fpgabased synthesis," *VLSI Design*, vol. 3, no. 3-4, pp. 289–300, 1995.
- [6] J. A. Brzozowski and T. Łuba, "Decomposition of boolean functions specified by cubes," *Journal of Multiple Valued Logic and Soft Computing*, vol. 9, no. 4, pp. 377–418, 2003.
- [7] A. Kuehlmann and F. Krohm, "Equivalence checking using cuts and heaps," in *Proceedings of the 34th annual Design Automation Conference*, 1997, pp. 263–268.
- [8] L. Amaru, P.-E. Gaillardon, and G. De Micheli, "Majority-inverter graph: A new paradigm for logic optimization," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, no. 5, pp. 806–819, 2015.
- [9] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill Education, 2019.
- [10] A. Mishchenko, R. Brayton, and S. Chatterjee, "Boolean factoring and decomposition of logic networks," in *2008 IEEE/ACM International Conference on Computer-Aided Design*. IEEE, 2008, pp. 38–44.
- [11] A. T. Calvino, G. De Micheli, A. Mishchenko, and R. Brayton, "Enhancing delay-driven lut mapping with boolean decomposition," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.
- [12] A. Mishchenko, R. Brayton, A. Tempia Calvino, and G. De Micheli, "Boolean decomposition revisited," in *International Logic Synthesis Workshop (IWLS)*, 2023.
- [13] A. Mishchenko, B. Steinbach, and M. Perkowski, "An algorithm for bi-decomposition of logic functions," in *Proceedings of the 38th annual design automation conference*, 2001, pp. 103–108.
- [14] M. Choudhury and K. Mohanram, "Bi-decomposition of large boolean functions using blocking edge graphs," in *2010 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2010, pp. 586–591.
- [15] V. Callegaro, F. S. Marranghello, M. G. Martins, R. P. Ribas, and A. I. Reis, "Bottom-up disjoint-support decomposition based on cofactor and boolean difference analysis," in *2015 33rd IEEE International Conference on Computer Design (ICCD)*. IEEE, 2015, pp. 680–687.
- [16] R. Brayton and A. Mishchenko, "Abc: An academic industrial-strength verification tool," in *Computer Aided Verification: 22nd International Conference, CAV 2010, Edinburgh, UK, July 15-19, 2010. Proceedings 22*. Springer, 2010, pp. 24–40.
- [17] R. Brayton, *Logic Minimization Algorithms for VLSI Synthesis*. Springer, 1984.
- [18] T. Mostafaie, F. M. Khiyabani, and N. J. Navimipour, "A systematic study on meta-heuristic approaches for solving the graph coloring problem," *Computers & Operations Research*, vol. 120, p. 104850, 2020.
- [19] M. Raasveldt and H. Mühleisen, "Duckdb: an embeddable analytical database," in *Proceedings of the 2019 International Conference on Management of Data*, 2019, pp. 1981–1984.
- [20] A. Hagberg, P. J. Swart, and D. A. Schult, "Exploring network structure, dynamics, and function using networkx," Los Alamos National Laboratory (LANL), Los Alamos, NM (United States), Tech. Rep., 2008.
- [21] F.-X. Reichl, F. Slivovsky, and S. Szeider, "Circuit minimization with qbf-based exact synthesis," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 4, 2023, pp. 4087–4094.

Device-Algorithm Co-Design of Ferroelectric Compute-in-Memory In-Situ Annealer for Combinatorial Optimization Problems

Yu Qian¹, Xianmin Huang¹, Ranran Wang¹, Zeyu Yang¹, Min Zhou¹, Thomas Kämpfe^{2,3}, Cheng Zhuo^{1,4*} and Xunzhao Yin^{1,4*}

¹Zhejiang University, Hangzhou, China; ²Fraunhofer IPMS, Dresden, Germany; ³TU Braunschweig, Braunschweig, Germany;

⁴Key Laboratory of CS&AUS of Zhejiang Province, Hangzhou, China; *Corresponding author email: {czhuo, xzyin1}@zju.edu.cn

ABSTRACT

Combinatorial optimization problems (COPs) are crucial in many applications but are computationally demanding. Traditional Ising annealers address COPs by directly converting them into Ising models (known as direct-E transformation) and solving them through iterative annealing. However, these approaches require vector-matrix-vector (VMV) multiplications with a complexity of $O(n^2)$ for Ising energy computation and complex exponential annealing factor calculations during annealing process, thus significantly increasing hardware costs. In this work, we propose a ferroelectric compute-in-memory (CiM) in-situ annealer to overcome aforementioned challenges. The proposed device-algorithm co-design framework consists of (i) a novel transformation method (first to our known) that converts COPs into an innovative incremental-E form, which reduces the complexity of VMV multiplication from $O(n^2)$ to $O(n)$, and approximates exponential annealing factor with a much simplified fractional form; (ii) a double gate ferroelectric FET (DG FeFET)-based CiM crossbar that efficiently computes the in-situ incremental-E form by leveraging the unique structure of DG FeFETs; (iii) a CiM annealer that approaches the solutions of COPs via iterative incremental-E computations within a tunable back gate-based in-situ annealing flow. Evaluation results show that our proposed CiM annealer significantly reduces hardware overhead, reducing energy consumption by 1503/1716 $\times$ and time cost by 8.08/8.15 $\times$ in solving 3000-node Max-Cut problems compared to two state-of-the-art annealers. It also exhibits high solving efficiency, achieving a remarkable average success rate of 98%, whereas other annealers show only 50% given the same iteration counts.

1 INTRODUCTION

Combinatorial optimization problems (COPs) have widespread applications across various fields, including logistics, resource allocation, communication network design, and transportation systems [1–4], and many of these problems are categorized as non-deterministic polynomial-time hard (NP-hard), reflecting their computational complexity. A wide range of COPs, including Max-Cut, graph coloring, and knapsack problems, etc., can be effectively reformulated into Ising models or Quadratic Unconstrained Binary Optimization (QUBO) forms, which can be addressed by Ising machines and QUBO solvers [5–8]. For example, digital ASIC annealers [9, 10] implement different annealing algorithms within digital circuits, and dynamical system Ising machines [11, 12] utilize the intrinsic system dynamics and their tendency to settle at ground state to solve COPs. These solvers address COPs more quickly and efficiently than von Neumann hardware, yet still suffer from significant hardware overhead and limited robustness.

Recently, numerous compute-in-memory (CiM) frameworks based on non-volatile memories (NVMs) such as ferroelectric field effect transistor (FeFET) and resistive random access memory (ReRAM), have been proposed to further improve the hardware performance in solving COPs, known as CiM annealers [7, 13–16]. Compared to the aforementioned solvers, CiM annealers utilize compact and

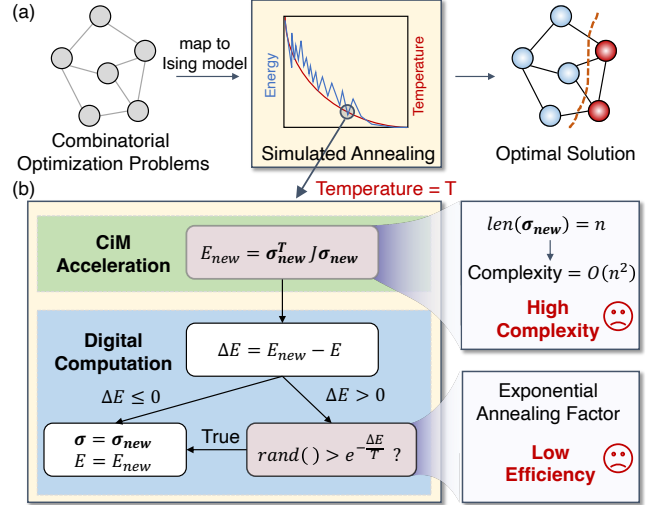


Figure 1: CiM annealer solves COPs. (a) General flow based on simulated annealing algorithm; (b) Current solvers have high complexity in CiM acceleration part, and low efficiency in digital computation part.

highly parallel NVM-based crossbar arrays to offer lower hardware costs and higher robustness. As Fig. 1(a) shows, after COPs are converted to a direct Ising model, CiM annealers compute the Ising energy within each simulated annealing iteration, progressively approaching optimal solutions as the temperature T decreases. As illustrated in Fig. 1(b), at a given T , the annealer computes the new Ising energy through a CiM accelerated vector-matrix-vector (VMV) multiplication, and calculates the incremental energy difference ΔE in the digital circuits. Depending on ΔE , an exponential annealing factor $e^{-\frac{\Delta E}{T}}$ is computed to decide whether to accept the new solution. Within the annealing process, CiM acceleration and digital computation are performed iteratively. As can be seen, the CiM part involves a complex VMV multiplication with a computational complexity of $O(n^2)$, leading to large energy consumption and increased time cost [17]. Additionally, the digital computation part requires calculating $e^{-\frac{\Delta E}{T}}$, which demands significant hardware resources, further reducing the computational efficiency [18].

To address aforementioned challenges, we introduce a device-algorithm co-design approach of ferroelectric CiM in-situ annealer that not only significantly reduces the complexity of direct Ising energy and exponential factor calculations, but also enhances the efficiency by leveraging the unique characteristics of a double gate ferroelectric FET (DG FeFET)-based CiM crossbar for in-situ annealing. The key contributions of this work are summarized as follows:

- We propose an incremental-E transformation method that converts a COP into an efficient incremental-E (E_{inc}) form. The E_{inc}

form eliminates redundant computations within the VMV multiplications, and approximates complex exponential annealing factor with simpler fractional annealing factor. As a result, the complexity of E_{inc} computation is reduced from $O(n^2)$ to $O(n)$, significantly alleviating the associated hardware overhead.

- We propose to realize a single DG FeFET-based four-input multiplication, as the device naturally supports the operator by applying three inputs to the double gates and drain, and storing an input as threshold voltage (V_{TH}) state, respectively. Based on the unique characteristic of devices, we propose a DG FeFET-based CiM crossbar array that computes the entire E_{inc} form, implementing the in-situ product of the VMV multiplication result and fractional annealing factor within a single array.
- Based on the proposed transformation method and the unique CiM design, we propose a CiM in-situ annealer, where the DG FeFET's back gate (BG) is utilized to mimic the temperature decay within the annealing flow due to its V_{TH} tunability. A tunable BG-based in-situ annealing flow is employed, performing all the analog computations within the CiM array, and leaving only the solution update in digital part.
- The proposed CiM annealer demonstrates significant reductions in both time and energy consumption while achieving high accuracy in solving COPs. In solving 30 Max-Cut problems with varying node numbers from 800 to 3000, it offers 410 $\times$ to 1706 $\times$ energy reductions and 7.98 $\times$ to 8.15 $\times$ time cost reductions compared to other two state-of-the-art annealers. It also achieves a 98% success rate in average, while other annealers reach only 50%, given the same iteration counts.

2 BACKGROUND

In this section, we first introduce the Ising models and related works, then review CiM and DG FeFET basics.

2.1 Ising models and existing solvers

Ising models provide a powerful framework for modeling and solving COPs [8], where problem variables are represented as spins with binary states, and constraints are captured through spin couplings. The objective function of the problem is formulated as the Hamiltonian energy function of the Ising model. Solving the problem involves finding the spin configuration that minimizes the corresponding Ising Hamiltonian H_P :

$$\min H_P = \sum_{i,j=1}^n J_{ij} \sigma_i \sigma_j + \sum_{i=1}^n h_i \sigma_i \quad (1)$$

where n denotes the number of spins and $\sigma_i \in \pm 1$ represents the state of spin i . J_{ij} and h_i stand for the coupling between spin i and j and the self-coupling of spin i , respectively. Eq. (1) can be expressed in an alternative general form by setting $J_{ii} = h_i$:

$$\min E = \sigma^T J \sigma \quad (2)$$

where σ presents the spin vector. The Ising model is equivalent to the QUBO form [19] through a simple variable transformation $\sigma_i = 1 - 2x_i$, $x_i \in \{0, 1\}$. Thus, the Ising machines and QUBO solvers can be regarded as equivalent COP solving designs.

Numerous Ising machines have been developed to address COPs. Digital ASIC annealers, which implement various annealing algorithms to solve Ising models [9, 10], often suffer from high energy consumption and time costs due to the exponential growth of explicit computations in each annealing iteration. Dynamic system Ising machines, on the other hand, exploit the natural dynamics of physical systems to explore optimal solutions [20, 21], but they

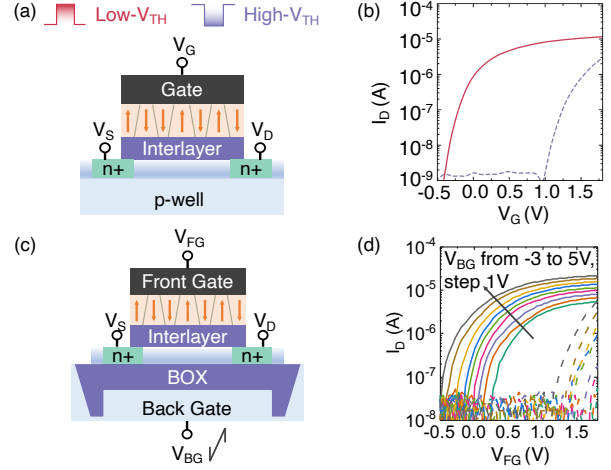


Figure 2: FeFET and DG FeFET. (a) A FeFET device with three terminals; (b) $I_D - V_G$ curves of a FeFET with programmed low/high V_{TH} . (c) A DG FeFET device with four terminals; (d) $I_D - V_G$ curves of the DG FeFET, where V_{BG} can flexibly adjust its V_{TH} after programming.

are highly sensitive to spin coupling implementations. Even slight deviations in coupling strength can hinder convergence [12, 20].

To further enhance the robustness and hardware efficiency of solving COPs, a novel framework known as the CiM annealer has been proposed recently [7, 14, 15]. Fig. 1(a) depicts the general flow of the CiM annealer. The COPs, e.g., Max-Cut problems, are formulated as Ising models. The Ising energy E is progressively minimized as temperature decreases, following the simulated annealing (SA) algorithm. At each temperature, the value of E_{new} based on updated spin states σ_{new} is computed using the CiM architecture, followed by peripheral digital computations. A FeFET-based CiM annealer in [7] provides a lossless compression method to reduce the cost of CiM architecture and enhances the SA algorithm into Multi-Epoch SA (MESA) for improved performance. The HyCiM framework in [15] addresses general COPs with inequality constraints by employing an inequality filter CiM structure. A C-Nash CiM framework is proposed to solve COPs with the MAX function embedded in the objective function [14]. While these CiM annealers leverage the CiM to reduce energy and time costs and enhance SA algorithm efficiency, they still face challenges on hardware overhead:

As shown in Fig. 1(b), the core VMV multiplication operation within each annealing iteration involves n^2 product terms given n spins, exhibiting a high complexity of $O(n^2)$. Additionally, as shown in Fig. 1(b), a complex exponential annealing factor calculation is required when $\Delta E > 0$, further reducing the efficiency of the CiM annealer. This has inspired us to propose an incremental-E transformation that reduces the complexity of VMV multiplication from $O(n^2)$ to $O(n)$, and eliminates the costly exponential function. Such incremental-E form is then implemented by a CiM array.

2.2 CiM Preliminaries and DG FeFET Basics

Prior CiM designs employ NVMs, such as ReRAM, magnetic tunneling junction (MTJ) and FeFET to build crossbars that store weight matrices, and perform parallel vector-matrix multiplication or VMV multiplication operations [7, 22–26]. Among these devices, FeFET offers advantages in energy and area efficiency due to the CMOS compatibility, voltage-driven mechanism, high ON/OFF ratio and

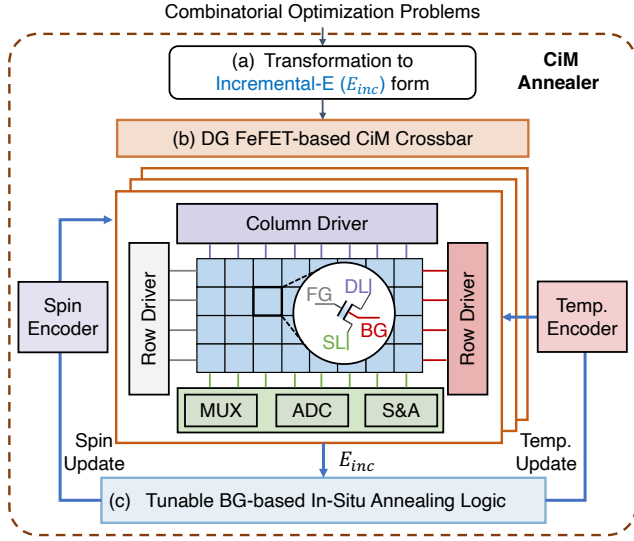


Figure 3: Overview of the proposed CiM annealer. (a) Incremental-E (E_{inc}) transformation method; (b) DG FeFET-based crossbar for E_{inc} computation; (c) Tunable BG-based in-situ annealing for approaching optimal solutions.

three-terminal structure [27–33]. When the gate is applied with different gate pulses as shown in Fig. 2(a), a FeFET can store high or low V_{TH} states, which are experimentally measured from the FeFET [7] as shown in Fig. 2(b). Prior works exploit the intrinsic three-input multiplication operator of FeFETs for VMV multiplication acceleration, thus enabling efficient CiM-based COP solvers [7, 14, 15]. That said, the potentials of ferroelectric devices are yet to be explored.

Recently, a novel technology known as DG FeFET [34] has been developed, which incorporates a non-ferroelectric (non-FE) dielectric in the BG. A notable example of such a device structure is fully-depleted silicon-on-insulator (FDSOI) FeFET, as shown in Fig. 2(c), where the buried oxide (BOX) layer serves as the non-ferroelectric dielectric. Unlike the fixed $I_D - V_G$ curves of a conventional FeFET, the $I_D - V_G$ curves for the low- V_{TH} /high- V_{TH} state of a DG FeFET can be dynamically adjusted by applying different voltage on BG, as illustrated in Fig. 2(d). This V_{TH} tuning does not change the ferroelectric states, but is rather due to the electric field exerted through the BG. This unique BG-based V_{TH} tuning is used in this work to create a cell that efficiently operates with three input terminals (i.e., double gates and drain), enabling in-situ annealing computations.

3 PROPOSED CIM ANNEALER

Here we introduce our proposed device-algorithm co-design approach, including the incremental-E transformation, the DG FeFET-based CiM array and tunable BG-based in-situ annealing flow.

3.1 Overview

Fig. 3 illustrates the overview of the proposed CiM annealer. The COP is firstly transformed into an incremental-E (E_{inc}) form (Sec. 3.2), which is then mapped onto the proposed DG FeFET-based CiM crossbar for computation (Sec. 3.3). Finally, the tunable BG-based in-situ annealing flow is employed, approaching optimal solutions (Sec. 3.4).

During each annealing iteration, the tunable BG-based annealing flow updates the spin states based on the previous E_{inc} value,

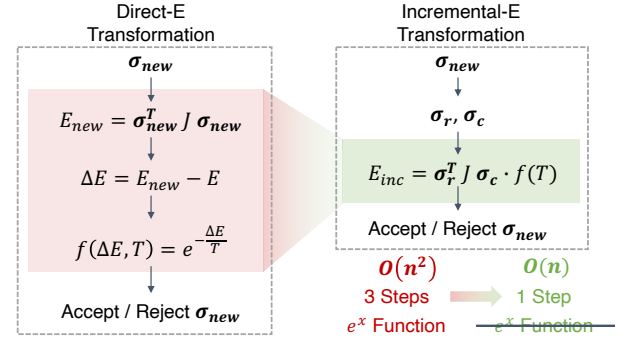


Figure 4: Compared to direct-E, the incremental-E transformation reduces complexity from $O(n^2)$ to $O(n)$, condenses three steps into a single operation, and eliminates e^x function.

transmitting solely the updated spins to the spin encoder. The updated temperature is relayed to the temperature encoder. The DG FeFET-based CiM array is then activated to compute the new E_{inc} , which is fed back to the annealing logic to initiate next iteration.

3.2 Incremental-E Transformation

Fig. 4 shows the proposed incremental-E transformation method, which replaces the costly VMV multiplication of E_{new} and e^x function in the direct-E transformation with the product of an incremental-based VMV multiplication and a fractional factor, thus reducing the complexity from $O(n^2)$ to $O(n)$. Such transformation merges the three critical computations into a single step, and eliminates the need for e^x calculation.

As shown in Fig. 1(b), current annealers calculate ΔE within an iteration as follows:

$$E_{new} = \sigma_{new}^T J \sigma_{new} \quad (3)$$

$$\Delta E = E_{new} - E \quad (4)$$

where E and E_{new} denote the current and new Ising energy, and σ and σ_{new} denote the current and new spin vectors, respectively. Fig. 5 shows the process of complexity reduction in the proposed incremental-E transformation. Within an iteration, the current spin vector σ (Fig. 5(a)) is flipped to generate σ_{new} (Fig. 5(b)). The corresponding E_{new} can be decomposed as:

$$E_{new} = \sum_{i,j \in \bar{F}} \sigma_i \sigma_j J_{ij} + \sum_{i,j \in F} \sigma_i \sigma_j J_{ij} + \sum_{i \in F, j \in \bar{F}} \sigma_i \sigma_j J_{ij} + \sum_{i \in \bar{F}, j \in F} \sigma_i \sigma_j J_{ij} \quad (5)$$

where each spin $\sigma_i \in \{-1, 1\}$, F denotes the index set of flipped spins, and $\bar{F}$ denotes the rest set. The computation of E_{new} entails n^2 product terms, exhibiting a complexity of $O(n^2)$. These terms can be categorized into three groups, i.e., $\sigma_i \sigma_j J_{ij}$ with 0/1/2 flipped spins, as shown in Fig. 5(b), corresponding to the boxes in blue/orange/green within the spin coupling matrix. If spin i and j are both flipped (switch to the other binary state), the term $\sigma_i \sigma_j J_{ij}$ remains unchanged. Therefore, the green terms can be recolored to blue, as depicted in Fig. 5(c). E_{new} can then be expressed as:

$$E_{new} = \sum_{(i,j \in F) \cup (i,j \in \bar{F})} \sigma_i \sigma_j J_{ij} + \sum_{(i \in \bar{F}, j \in F) \cup (i \in F, j \in \bar{F})} \sigma_i \sigma_j J_{ij} \quad (6)$$

where the latter term in orange corresponds to the distinct terms between E and E_{new} . These orange terms can be merged along the diagonal of matrix due to the symmetry. After the merging, we define a logical vector σ_f , where $\sigma_{f,i} = 1$ indicates that spin i is flipped. We also define two vectors, σ_c and σ_r , based on σ_{new} :

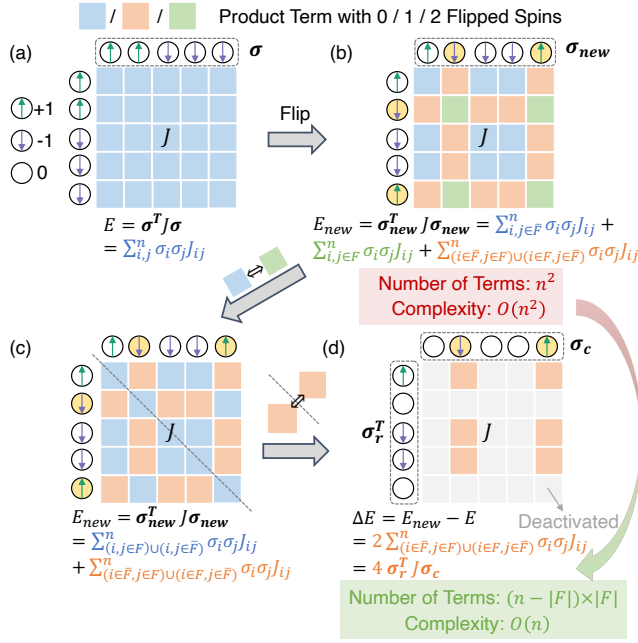


Figure 5: Complexity reduction of VMV multiplication in the incremental-E transformation. After (a) σ is flipped to (b) σ_{new} , E_{new} has a complexity of $O(n^2)$, composing of three groups of product terms in different colors; (c) The distinct terms between E and E_{new} in orange are merged by diagonal symmetry, resulting in (d) ΔE with $O(n)$ complexity.

$$\sigma_c = \sigma_{new} \circ \sigma_f \quad (7)$$

$$\sigma_r = \sigma_{new} \circ (1 - \sigma_f) \quad (8)$$

where $\circ$ denotes element-wise multiplication. The vector σ_c maintains the values of the flipped elements in σ_{new} , setting all others to zero, while σ_r preserves the unflipped spin values, zeroing out the flipped elements. As a result, ΔE can be expressed as an incremental-based VMV multiplication:

$$\Delta E = 4\sigma_r^T J \sigma_c \quad (9)$$

where the number of product terms is only $(n - |F|) \times |F|$, exhibiting a reduced complexity of $O(n)$ because $|F|$ is set to be constant, as depicted in Fig. 5(d).

The exponential annealing factor in Fig. 1(b) is approximated as

$$e^{-\frac{\Delta E}{T}} \approx 1 - \frac{\Delta E}{T} \propto -\frac{\Delta E}{T} \quad (10)$$

Therefore, after simplification, the incremental-E form is as follows:

$$E_{inc} = \sigma_r^T J \sigma_c \cdot f(T) \quad (11)$$

where $f(T) = \frac{a}{bT+c} + d$, with parameters a , b , c , and d properly chosen. Note that Eq. (11) must satisfy (i) $f(T) > 0$, as $e^{-\frac{\Delta E}{T}} > 0$, and (ii) $f(T) \propto T$. The resulted E_{inc} formulation can be efficiently implemented by our proposed DG FeFET-based CiM crossbar.

3.3 DG FeFET-based CiM Crossbar

As shown in Fig. 6(a), a single DG FeFET has four terminals: the data line (DL), front gate (FG), back gate (BG) and source line (SL). A low/high V_{TH} is stored in the DG FeFET, denoted as $G = '1'/'0'$. The inputs at FG, DL and BG are denoted as x , y and z , where x and y are binary inputs ($x/y \in \{0, 1\}$) and z is an analog input. SL is the output terminal. Fig. 6(b) shows the $I_{SL} - V_{BG}$ characteristics given fixed DL and FG inputs. It can be seen that DG FeFET naturally performs the operation $I_{SL} = x \cdot G \cdot y \cdot z$, where I_{SL} remains 0 when

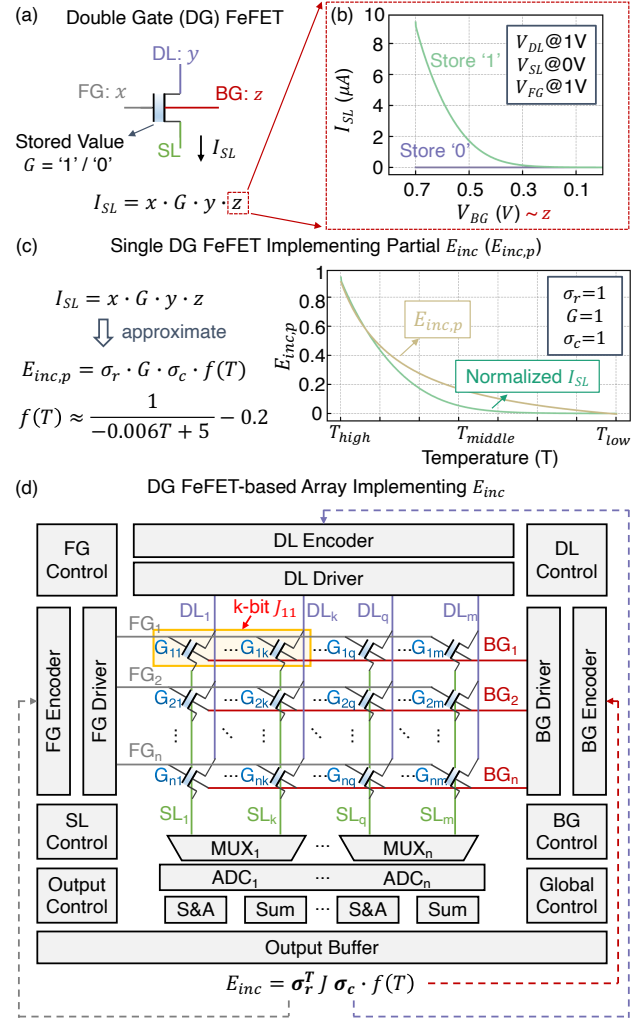


Figure 6: DG FeFET-based CiM crossbar array. (a) A DG FeFET naturally performs $I_{SL} = x \cdot G \cdot y \cdot z$; (b) V_{BG} can adjust the V_{TH} , thus correspondingly tune I_{SL} with DG FeFET storing '1'; (c) $I_{SL} - V_{BG}$ of a DG FeFET can approximate the partial incremental-E form $E_{inc,p}$, where the fractional annealing factor $f(T)$ is mapped as I_{SL} ; (d) The DG FeFET-based crossbar array can implement incremental-E form E_{inc} .

$G = '0'$, and follows V_{BG} when storing '1'. This intrinsic operation of the DG FeFET resembles the partial incremental-E formulation, i.e., $E_{inc,p} = \sigma_r \cdot G \cdot \sigma_c \cdot f(T)$. As illustrated in Fig. 6(c), the normalized output current I_{SL} (with '1' stored) can approximate $E_{inc,p}$ assuming $f(T) \approx \frac{1}{-0.006T+5} - 0.2$, where G is stored as binary V_{TH} state, σ_r , σ_c are applied to FG, DL, respectively, and T corresponds to V_{BG} . I_{SL} is sensed as the value of $E_{inc,p}$.

Fig. 6(d) shows the schematic of the proposed DG FeFET-based crossbar array. Each row and column share the same FG/BG and DL/SL, respectively. The peripheral circuits include controllers, encoders and drivers for FG, DL and BG inputs, and multiplexers (MUXs), ADCs, shift&adders (S&As), summations (Sums) and an output buffer for SL outputs. Every k columns share the same MUX, ADC, S&A and Sum. Each element in the matrix J , e.g., J_{11} , is mapped onto a $1 \times k$ subarray, with each cell storing 1 bit under

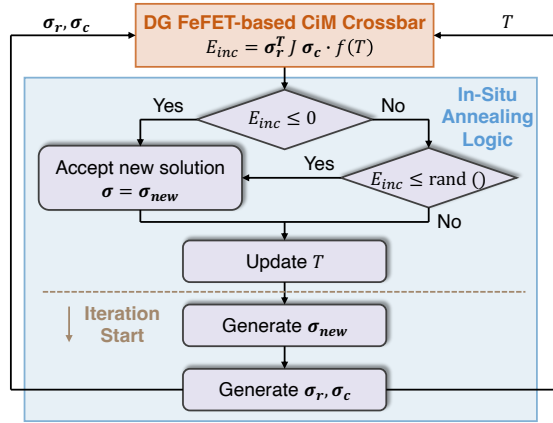


Figure 7: Working flow of the proposed CiM annealer.

k -bit quantization. In this way, an $n \times n$ matrix J is mapped onto an $n \times m$ crossbar ($m = n \times k$).

During each E_{inc} computation, the input vectors σ_r^T and σ_c are simultaneously directed to FG and DL encoders, respectively, and the input scalar $f(T)$ is sent to the BG encoder. FG and DL encoders convert σ_r^T and σ_c into binary voltages and drive the array through FG and DL drivers, respectively. The BG encoder looks at T value and generates the corresponding analog voltage via the BG driver to control all BGs. For the array output, k SLs form a group, and all groups perform the computations in parallel. k output currents from a group are sensed sequentially, multiplexed by the MUX and converted into k digital values via the ADC. The k digital values are then shifted, added and summed to generate the partial result. These partial results from all groups are aggregated to compute the final E_{inc} value, stored in the output buffer. Since the crossbar only supports non-negative inputs, the E_{inc} components associated with positive and negative inputs are separately calculated, respectively, and summed ultimately. To this end, the DG FeFET-based crossbar array successfully implements $E_{inc} = \sigma_r^T J \sigma_c \cdot f(T)$.

3.4 Tunable BG-based In-Situ Annealing Flow

Based on the incremental-E transformation and DG FeFET-based CiM crossbar array, we present the proposed CiM annealer that leverages the V_{TH} tunable BG of DG FeFETs for efficient in-situ annealing process. Fig. 7 illustrates the work flow of the annealer. In each annealing iteration, a new solution σ_{new} is randomly generated based on the current solution σ . The vectors σ_r and σ_c are derived from σ_{new} . The σ_r , σ_c and temperature T are then directed to the DG FeFET-based CiM crossbar for in-situ E_{inc} computation. The computed result of E_{inc} is sent to the annealing logic. If $E_{inc} \leq 0$, it indicates that the new solution corresponds to lower energy compared to current solution, thus the new solution is accepted, updating the current solution $\sigma = \sigma_{new}$. If $E_{inc} > 0$, the value is compared with a randomly generated number $r \in [0, 1]$. If $E_{inc} \leq r$, the new solution is still accepted; otherwise, σ remains unchanged. The current solution σ then serves as the input of next iteration and the temperature T is updated. The optimal solution is identified through iterative computations. Alg. 1 presents the pseudocode for the annealing algorithm.

The range of annealing temperature T is normalized to match the BG voltage range V_{BG} (0.7V to 0V) for the DG FeFET-based crossbar with a gradient of 0.01V, such that the temperature related annealing

Algorithm 1 In-Situ Annealing Algorithm

Input: J, t $\triangleright$ Ising model and number of flipped spins ($t = |F|$)
Output: σ $\triangleright$ Spin configuration that minimizes $E = \sigma^T J \sigma$

- 1: Initialize σ with random values from $\{-1, 1\}$
- 2: **for** each annealing iteration **do**
- 3: Select t elements and get σ_f $\triangleright$ Marks spins to flip
- 4: $\sigma_{new} = \sigma \circ (1 - 2 \cdot \sigma_f)$ $\triangleright$ Flip elements in σ
- 5: $\sigma_c = \sigma_{new} \circ \sigma_f, \sigma_r = \sigma_{new} \circ (1 - \sigma_f)$
- 6: $E_{inc} = \sigma_r^T J \sigma_c \cdot f(T)$ $\triangleright$ DG FeFET-based CiM acceleration
- 7: **if** $E_{inc} \leq 0$ **then**
- 8: $\sigma = \sigma_{new}$ $\triangleright$ Accept new solution
- 9: **else**
- 10: **if** $E_{inc} \leq \text{random}(0, 1)$ **then**
- 11: $\sigma = \sigma_{new}$ $\triangleright$ Accept new solution
- 12: **end if**
- 13: **end if**
- 14: Update T based on annealing schedule
- 15: **end for**

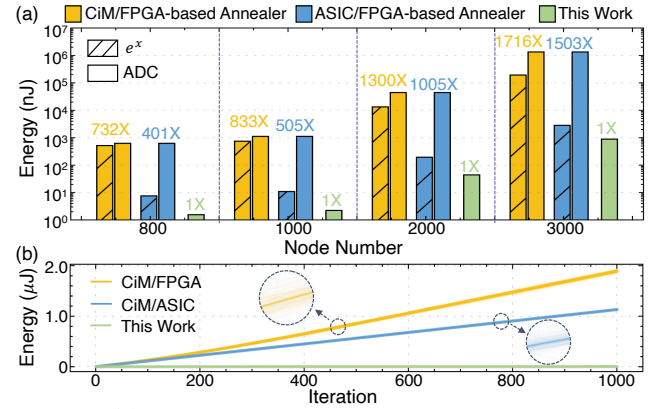


Figure 8: (a) Average energy in solving 30 Max-Cut problem instances and (b) energy trends in solving a 1000-node instance across three CiM annealers.

factor $f(T)$ can be approximately mapped to the crossbar currents by tuning V_{BG} , as illustrated in Fig. 6(c). T does not necessarily decrease with each iteration. For longer processes, T decreases only after a pre-set number of iterations. Once V_{BG} reaches 0V, it remains at zero, terminating the annealing.

4 EVALUATION

In this section, we first compare the hardware overhead of the proposed CiM annealer to other two CiM annealers [7, 18]. We then evaluate their efficiency in solving COPs. All simulations were conducted using SPECTRE. The 22nm BSIM IMG DG FeFET model [34] and Preisach FeFET model [35] are adopted for DG FeFET and FeFET, respectively. A 22nm commercial model is used for MOSFETs with TT process corner at a temperature of 27°C. Each annealer contains a single crossbar. 8-to-1 multiplexed ADCs are employed [36], which are scaled to 22nm. The hardware overhead for the exponential function implementation is derived from [18]. The wiring parasitics are extracted from DESTINY [37]. The Max-Cut problem is selected as the representative COP [38].

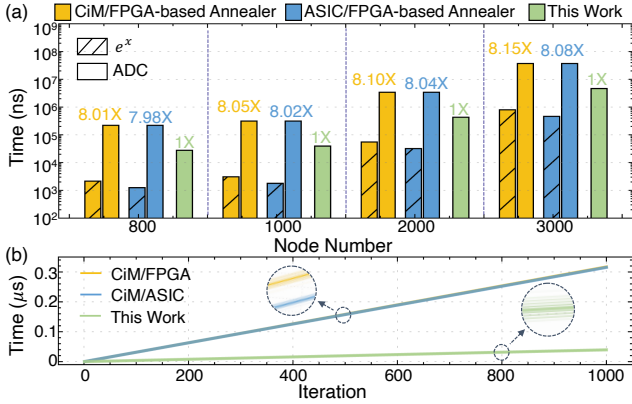


Figure 9: (a) Average time cost in solving 30 Max-Cut problems and (b) time cost trends in solving a 1000-node instance across three CiM annealers.

4.1 Hardware Overhead

For a fair comparison, the other two annealers are based on FeFET CiM accelerations [7] and FPGA/ASIC modules for exponential computation [18], denoted as CiM/FPGA and CiM/ASIC-based annealers, respectively. We selected 30 Max-Cut problem instances across four groups: 9 with 800 nodes, 9 with 1000 nodes, 9 with 2000 nodes, and 3 with 3000 nodes. For each instance, we conducted 100 runs. The iteration numbers per run are set according to problem size, with 700/1000/10,000/100,000 iterations for the 800-/1000-/2000-/3000-node instances, respectively.

Fig. 8(a) illustrates the average energy consumption of the three annealers. The major energy consumption are from the ADC and the exponential function implementation. It can be seen that, our proposed in-situ annealer achieves significant energy savings by reducing the ADC execution times and eliminating the energy associated with e^x calculation. This efficiency stems from the incremental-E transformation, which removes redundant VMV multiplications and simplifies the exponential calculation to a fractional function. Such transformation fully takes advantages of the DG FeFET-based CiM array, enabling efficient in-situ E_{inc} computations. Notably, the energy efficiency improvement increases as the problem size grows. Fig. 8(b) shows the energy consumptions versus iteration count of the three annealers when solving a 1000-node instance. The CiM/FPGA and CiM/ASIC-based annealers display a rapid linear energy increase with iteration counts. Conversely, our proposed annealer demonstrates a significantly slower rise in energy consumption. This is because larger-scale COPs require more annealing iterations, enlarging the energy savings associated with reduced ADC executions and simplified fractional factor calculations.

Fig. 9(a) presents the average time for problem solving. The proposed annealer shows substantial time savings across different problem sizes. Unlike prior annealers that activate the entire array for energy computation, the proposed annealer only activates specific columns associated with updated spins, thus significantly reducing the overall ADC time cost. Fig. 9(b) shows the time cost trends in solving a 1000-node instance. The other two annealers have identical trends due to the dominant ADC time. The proposed annealer shows much less time, as it eliminates significant ADC executions associated with redundant energy form computations.

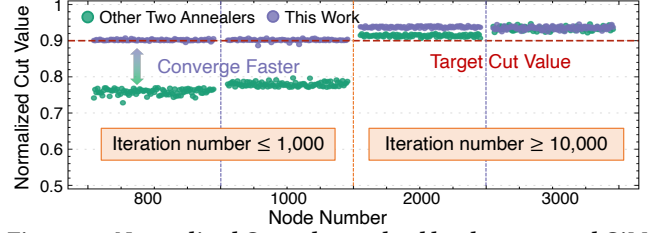


Figure 10: Normalized Cut values solved by the proposed CiM in-situ annealer and other annealers.

Table 1: Summary of COP Solvers

Reference	[39]	[7]	[13]	[15]	[14]	This Work
COP	Max-Cut	Graph Coloring	Knapsack	Quadratic Knapsack	Nash Equilibrium	Max-Cut
Complexity	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n)$
e^x Computation	Yes	Yes	Yes	Yes	Yes	No
Hardware Imp.* for Crossbar	Memristor	FeFET	RRAM	FeFET	FeFET	DG FeFET
Problem Size	60 node	21 node	10 node	100 node	104 node	3000 node
Time to Solution	6.6 μ s	5.1 μ s [‡]	3.8 μ s [‡]	1.3ms [‡]	0.08s	4.6ms
Energy to Solution	0.07 μ J	0.2 μ J [‡]	-	2.1 μ J [‡]	-	0.9 μ J
Average Success Rate (%)	65 [†]	-	92.4 [†]	98.54	81.9	98

*: Imp.: implementation. †: Extracted from literature. ‡: Extracted from literature and estimation at 22nm node.

4.2 Problem Solving Efficiency

We compare the COP solving efficiency of the three CiM annealers. Because CiM/FPGA and CiM/ASIC-based annealers adopt the same algorithm, the results are consistent. We used Monte Carlo sampling to select 100 runs for each Max-Cut problem group, and the normalized Cut values are shown in Fig. 10. The target Cut value is set as 90% of the true optimal value, suggesting a successful instance solving. The results clearly demonstrate that the proposed annealer consistently approaches the optimal solution for all COPs, achieving an impressive average success rate of 98%. In contrast, the other two annealers show a lower average success rate of 50%, only solving the 2000-node and 3000-node problems successfully. Interesting, the fractional annealing factor approximately implemented by DG FeFET's normalized current enables faster convergence than other annealers with exponential annealing factor, achieving higher solving efficiency. The solver summary in Table. 1 shows that the proposed CiM in-situ annealer can solve larger COPs with high energy and time efficiency, achieving a high success rate.

5 CONCLUSION

In this paper, we introduce a ferroelectric CiM in-situ annealer to efficiently solve COPs. We present an incremental-E transformation method that converts a COP into an E_{inc} form, reducing computational complexity and eliminating costly exponential function calculations. We propose a DG FeFET-based CiM crossbar to implement the E_{inc} form by utilizing the unique characteristic and structure of DG FeFETs. Overall, we build a device-algorithm co-design framework that accommodates efficient in-situ E_{inc} computation and annealing flow to solve COPs. Evaluation results show that the proposed annealer significantly reduces energy consumption and time costs while improving the problem solving efficiency compared to prior counterpart solvers.

ACKNOWLEDGMENT

This work was partially supported by NSFC (624B2126, W2412034, 92164203) and SGC Cooperation Project (Grant No. M-0612).

REFERENCES

- [1] G. Yu, *Industrial applications of combinatorial optimization*. Springer Science & Business Media, 2013, vol. 16.
- [2] V. T. Paschos, *Applications of combinatorial optimization*. John Wiley & Sons, 2014, vol. 3.
- [3] G. Naseri and M. A. Koffas, "Application of combinatorial optimization strategies in synthetic biology," *Nature communications*, vol. 11, no. 1, p. 2446, 2020.
- [4] F. Barahona, M. Grötschel, M. Jünger *et al.*, "An application of combinatorial optimization to statistical physics and circuit layout design," *Operations Research*, vol. 36, no. 3, pp. 493–513, 1988.
- [5] M. Jiang, K. Shan, C. He *et al.*, "Efficient combinatorial optimization by quantum-inspired parallel annealing in analogue memristor crossbar," *Nature Communications*, vol. 14, no. 1, p. 5927, 2023.
- [6] W. Yue, T. Zhang, Z. Jing *et al.*, "A scalable universal ising machine based on interaction-centric storage and compute-in-memory," *Nature Electronics*, vol. 7, no. 10, pp. 904–913, 2024.
- [7] X. Yin, Y. Qian, A. Vardar *et al.*, "Ferroelectric compute-in-memory annealer for combinatorial optimization problems," *Nature Communications*, vol. 15, no. 1, p. 2419, 2024.
- [8] N. Mohseni, P. L. McMahon, and T. Byrnes, "Ising machines as hardware solvers of combinatorial optimization problems," *Nature Reviews Physics*, vol. 4, pp. 363–379, 2022.
- [9] K. Katsuki, D. Shin, N. Onizawa *et al.*, "Fast solving complete 2000-node optimization using stochastic-computing simulated annealing," in *2022 29th IEEE International Conference on Electronics, Circuits and Systems (ICECS)*. IEEE, 2022, pp. 1–4.
- [10] T. Takemoto, K. Yamamoto, C. Yoshimura *et al.*, "4.6 a 144kb annealing system composed of 9× 16kb annealing processor chips with scalable chip-to-chip connections for large-scale combinatorial optimization problems," in *2021 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 2021, pp. 64–66.
- [11] W. Moy, I. Ahmed, P.-w. Chiu *et al.*, "A 1,968-node coupled ring oscillator circuit for combinatorial optimization problem solving," *Nature Electronics*, vol. 5, no. 5, pp. 310–317, 2022.
- [12] I. Ahmed, P.-W. Chiu, W. Moy *et al.*, "A probabilistic compute fabric based on coupled ring oscillators for solving combinatorial optimization problems," *IEEE Journal of Solid-State Circuits*, vol. 56, no. 9, pp. 2870–2880, 2021.
- [13] K. Taoka, N. Misawa, S. Koshino *et al.*, "Simulated annealing algorithm & reram device co-optimization for computation-in-memory," in *2021 IEEE International Memory Workshop (IMW)*. IEEE, 2021, pp. 1–4.
- [14] Y. Qian, K. Ni, T. Kampfe *et al.*, "C-nash: A novel ferroelectric computing-in-memory architecture for solving mixed strategy nash equilibrium," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
- [15] Y. Qian, Z. Yang, K. Ni *et al.*, "Hycim: A hybrid computing-in-memory qubo solver for general combinatorial optimization problems with inequality constraints," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
- [16] Y. Qian, D. Huang, A. Vardar *et al.*, "Ferroelectric compute-in-memory framework for solving pure and mixed strategy nash equilibrium," *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2025.
- [17] S. Kim, S. Um, W. Jo *et al.*, "Scaling-cim: edram in-memory-computing accelerator with dynamic-scaling adc and adaptive analog operation," *IEEE Journal of Solid-State Circuits*, 2024.
- [18] M. A. Hussain, S.-W. Lin, and T.-H. Tsai, "An area-efficient and high throughput hardware implementation of exponent function," in *2022 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2022, pp. 3369–3372.
- [19] F. Glover, G. Kochenberger, and Y. Du, "A tutorial on formulating and using qubo models," *arXiv preprint arXiv:1811.11538*, 2018.
- [20] A. Mallick, Z. Zhao, M. K. Bashir *et al.*, "Cmos-compatible ising machines built using bistable latches coupled through ferroelectric transistor arrays," *Scientific reports*, vol. 13, p. 1515, 2023.
- [21] R. Afoakwa, Y. Zhang, U. K. R. Vengalam *et al.*, "Brim: bistable resistively-coupled ising machine," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 749–760.
- [22] C. Zhuo, Z. Yang, K. Ni *et al.*, "Design of ultra-compact content addressable memory exploiting 1t-1mtj cell," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2022.
- [23] Y. Qian, L. Zhao, F. Meng *et al.*, "Enhancing convnets with convfio: A crossbar pim architecture based on kernel-stationary first-in-first-out dataflow," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, no. 01, pp. 1–12, 2024.
- [24] L. Zhao, Y. Qian, F. Meng *et al.*, "Convffio: A crossbar memory pim architecture for convnets featuring first-in-first-out dataflow," in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2024, pp. 824–829.
- [25] A. Mondal and A. Srivastava, "In-situ stochastic training of mtj crossbar based neural networks," in *Proceedings of the International Symposium on Low Power Electronics and Design*, 2018, pp. 1–6.
- [26] X. Yin, Q. Huang, H. E. Barkam *et al.*, "A homogeneous fefet-based time-domain compute-in-memory fabric for matrix-vector multiplication and associative search," *IEEE TCAD*, 2024.
- [27] K. Ni, X. Yin, A. F. Laguna *et al.*, "Ferroelectric ternary content-addressable memory for one-shot learning," *Nature Electronics*, vol. 2, no. 11, pp. 521–529, 2019.
- [28] H. Xu, J. Yang, T. Kämpfe *et al.*, "On the challenges and design mitigations of single transistor ferroelectric content addressable memory," *IEEE EDL*, 2023.
- [29] J. Cai, H. E. Barkam, M. Imani *et al.*, "A scalable 2t-1fefet based content addressable memory design for energy efficient data search," *IEEE TCAD*, 2024.
- [30] X. Yin, F. Müller, Q. Huang *et al.*, "An ultracompact single-ferroelectric field-effect transistor binary and multibit associative search engine," *Advanced Intelligent Systems*, vol. 5, no. 7, p. 2200428, 2023.
- [31] X. Yin, F. Müller, A. F. Laguna *et al.*, "Deep random forest with ferroelectric analog content addressable memory," *Science Advances*, vol. 10, p. eadk8471, 2024.
- [32] X. S. Hu, M. Niemier, A. Kazemi *et al.*, "In-memory computing with associative memories: A cross-layer perspective," in *2021 IEEE International Electron Devices Meeting (IEDM)*. IEEE, 2021, pp. 25–2.
- [33] X. Yin, Y. Qian, M. Imani *et al.*, "Ferroelectric ternary content addressable memories for energy-efficient associative search," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 4, pp. 1099–1112, 2022.
- [34] Z. Jiang, Y. Xiao, S. Chatterjee *et al.*, "Asymmetric double-gate ferroelectric fet to decouple the tradeoff between thickness scaling and memory window," in *2022 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. IEEE, 2022, pp. 395–396.
- [35] K. Ni, M. Jerry, J. A. Smith *et al.*, "A circuit compatible accurate compact model for ferroelectric-fets," in *2018 IEEE symposium on VLSI technology*. IEEE, 2018, pp. 131–132.
- [36] J. Liu, X. Tang, W. Zhao *et al.*, "16.5 a 13b 0.005 mm 2 40ms/s sar adc with kt/c noise cancellation," in *2020 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 2020, pp. 258–260.
- [37] M. Poremba, S. Mittal, D. Li *et al.*, "Destiny: A tool for modeling emerging 3d nvm and edram caches," in *2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2015, pp. 1543–1546.
- [38] "Stanford Max-Cut dataset," <https://web.stanford.edu/~yyye/yyye/Gset/>.
- [39] F. Cai, S. Kumar, T. Van Vaerenbergh *et al.*, "Power-efficient combinatorial optimization using intrinsic noise in memristor hopfield neural networks," *Nature Electronics*, vol. 3, no. 7, pp. 409–418, 2020.